

Figure 4.20. The kernel and device drivers sit directly on top of the hardware, and run in privileged mode. All other operating system software and all user programs are implemented on top of the kernel and driver layer, and run in a somewhat restricted user mode.

- cloud computing.

We'll focus exclusively on parallelism within a single computer in this book, but you can read more about distributed computing by searching for the above terms online.

4.4 Operating System Fundamentals

Now that we have a solid understanding of the basics of parallel computer hardware, let's turn our attention to the services provided by the operating system that make concurrent programming possible.

4.4.1 The Kernel

Modern operating systems handle a wide variety of tasks, across a wide range of granularities. These range from the handling of keyboard and mouse events or the scheduling of programs for *preemptive multitasking* at one end of the spectrum, to managing a printer queue or the network stack at the other end. The “core” of the operating system—the part that handles all of the most fundamental and lowest-level operations—is called the *kernel*. The rest of the operating system, and all user programs, are built atop the services provided by the kernel. This architecture is illustrated in Figure 4.20.

4.4.1.1 Kernel Mode versus User Mode

The kernel and its device drives run in a special mode called *protected mode*, *privileged mode* or *kernel mode*, while all other programs in the system (including all other parts of the operating system that aren't part of the kernel) operate in *user mode*. As the name suggests, software running in privileged mode has

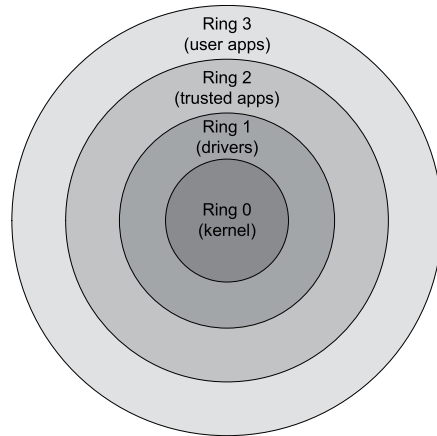


Figure 4.21. Example of CPU protection rings, showing four rings. The kernel runs in ring 0, device drivers run in ring 1, trusted programs with I/O permissions run in ring 2, and all other user programs run in ring 3.

full access to all of the hardware in the computer, whereas user mode software is *restricted* in various ways in order to ensure stability of the computer system as a whole. Software running in user mode can access low-level services only by making a special *kernel call*—a request for the kernel to perform a low-level operation on the user program’s behalf. This ensures that a program can’t inadvertently or maliciously destabilize the system.

In practice, operating systems may implement multiple *protection rings*. The kernel runs in ring 0, which is the most trusted ring and has all possible privileges within the system. Device drivers might run in ring 1, trusted programs with I/O permissions might run in ring 2, while all other “untrusted” user programs run in ring 3. But this is just one example—the number of rings varies from CPU to CPU and from OS to OS, as does the assignment of subsystems to the various rings. The protection ring concept is illustrated in Figure 4.21.

4.4.1.2 Kernel Mode Privileges

Kernel mode (ring 0) software has access to *all* of the machine language instructions defined by the CPU’s ISA. This includes a powerful subset of instructions called *privileged* instructions. These privileged instructions might allow certain registers to be modified that are normally off-limits (e.g., to control virtual memory mapping, or to mask and unmask interrupts). Or they might allow certain regions of memory to be accessed, or allow other nor-

mally restricted operations to be performed. Examples of privileged instructions on the Intel x86 processor include `wrmsr` (write to model-specific register) and `cli` (clear interrupts). By restricting the use of these powerful instructions only to “trusted” software like the kernel, system stability and security is improved.

With these privileged ML instructions, the kernel can implement security measures. For example, the kernel typically locks down certain pages of virtual memory so that they cannot be written to by a user program. Both the kernel’s software and all of its internal record-keeping data are kept in protected memory pages. This ensures that a user program won’t stomp on the kernel, and thereby crash the entire system.

4.4.2 Interrupts

An *interrupt* is a signal sent to the CPU in order to notify it of an important low-level event, such as a keypress on the keyboard, a signal from a peripheral device, or the expiration of a timer. When such an event occurs, an *interrupt request* (IRQ) is raised. If the operating system wishes to respond to the event, it pauses (interrupts) whatever processing had been going on, and calls a special kind of function called an *interrupt service routine* (ISR). The ISR function performs some operation in response to the event (ideally doing so as quickly as possible) and then control is returned back to whatever program had been running prior to the interrupt having been raised.

There are two kinds of interrupt: *hardware interrupts* and *software interrupts*. A hardware interrupt is requested by placing a non-zero voltage onto one of the pins of the CPU. Hardware interrupts might be raised by devices such as a keyboard or mouse, or by a periodic timer circuit on the motherboard or within the CPU itself. Because it’s triggered by an external device, a hardware interrupt can happen at any time—even right in the middle of executing a CPU instruction. As such, there may be a tiny delay between the moment when a hardware interrupt is physically raised and when the CPU is in a suitable state to handle it.

A software interrupt is triggered by software rather than by a voltage on a CPU pin. It has the same basic effect as a hardware interrupt, in that it causes the operation of the CPU to be interrupted and a service routine to be called. A software interrupt can be triggered explicitly by executing an “interrupt” machine language instruction. Or one may be triggered in response to an erroneous condition detected by the CPU while running a piece of software—these are called *traps* or sometimes *exceptions* (although the latter term should not be confused with language-level exception handling). For example, if an ALU is

instructed to perform a divide-by-zero operation, a software interrupt will be raised. The operating system normally handles such interrupts by crashing the program in question and producing a *core dump* file. However, a debugger attached to the program could catch this interrupt and instead cause the program to break into the debugger for inspection.

4.4.3 Kernel Calls

In order for user software to perform a privileged operation, such as mapping or unmapping physical memory pages in the virtual memory system or accessing a raw network socket, the user program must make a request to the kernel. The kernel responds by performing the operation in a safe manner on behalf of the user program. Such a request is called a *kernel call* or *system call*.

On most systems, calling into the kernel is accomplished via a software interrupt.² In the case of an interrupt-triggered system call, the user program places any input arguments in a specific place (either in memory or in registers) and then issues a “software interrupt” instruction with an integer argument that specifies which kernel operation is being requested. This causes the CPU to be put into a mode with elevated privileges, staves the state of the calling program, and then causes the appropriate kernel interrupt service routine to be called. Presuming that the kernel allows the request to proceed, it performs the requested operation (in privileged mode) and then control is returned to the caller (after first restoring its execution state). This switch from a user mode program into the kernel is an example of *context switching*. See Section 4.4.6.5 for more on context switching.

On most modern operating systems, the user program doesn’t execute a software interrupt or system call instruction manually, for example via inline assembly code. That would be messy and error-prone. Instead, a user program calls a *kernel API* function, which in turn marshalls the arguments and trips the software interrupt. This is why system calls appear to be regular function calls from the point of view of the user program.

4.4.4 Preemptive Multitasking

The earliest minicomputers and personal computers ran a single program at a time. They were inherently *serial* computers, capable of reading a program from a single instruction stream, and executing one instruction from this stream at a time. The *disk operating systems* (DOS) in those days weren’t much

²On some systems, a special variant of the `call` instruction is used to call into the kernel. For example, on MIPS processors this instruction is named `syscall`.

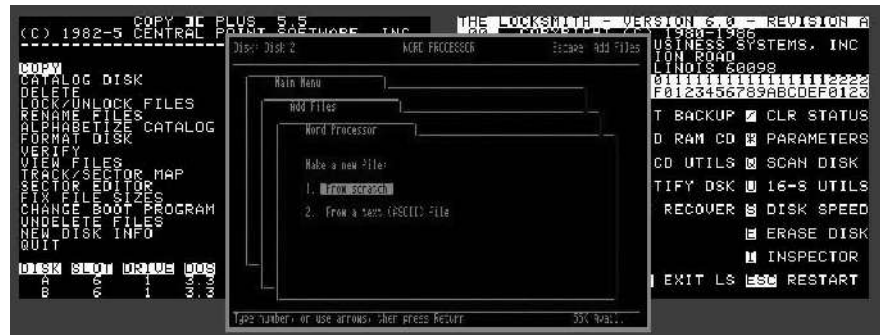


Figure 4.22. Three full-screen programs running on the Apple II computer. Programs on the Apple II were always full-screen because it could only run one program at a time. From left to right: Copy II Plus, the AppleWorks word processor, and The Locksmith.

more than glorified device drivers, allowing programs to interface with devices such as tape, floppy and hard disk drives. The entire computer would be devoted to running a single program at a time. Figure 4.22 shows a few full-screen programs running on an Apple II computer.

As operating systems and computer hardware became more advanced, it became possible to run more than one program on a serial computer at a time. On shared mainframe computer systems, a technique known as *multiprogramming* would allow one program to run while another was waiting for a time-consuming request to be satisfied by a peripheral device. Classic Mac OS and versions of Windows prior to Windows NT and Windows 95 used a technique known as *cooperative multitasking*, in which only one program would be running on the machine at a time, but each program would regularly *yield* the CPU so that another program could get a chance to run. In this way, each program ended up with a periodic “slice” of CPU time. Technically, this technique is known as *time division multiplexing* (TDM) or *temporal multithreading* (TMT). Informally it’s called *time-slicing*.

Cooperative multitasking suffered from one big problem: Time-slicing required the cooperation of each and every program in the system. One “rogue” program could consume all of the CPU’s time if it failed to yield to other programs periodically. The PDP-6 Monitor and Multics operating systems solved this problem by introducing a technique known as *preemptive multitasking*. This technology was later adopted by the UNIX operating system and all of its variants, along with later versions of Mac OS and Windows.

In preemptive multitasking, programs still share the CPU by time-slicing. However the scheduling of programs is controlled by the operating system, not via cooperation between the programs themselves. As a result, each pro-

gram gets a regular, consistent and reliable time slice on the CPU. The time slice during which one particular program is allowed to run on the CPU is sometimes called the program's *quantum*. To implement preemptive multitasking, the operating system responds to a regularly-timed hardware interrupt in order to periodically *context switch* between the different programs running on the system. We'll see how context switching works in more depth in the next section (4.4.6.5).

We should note here that preemptive multitasking is used even on multicore machines, because typically the number of threads is greater than the number of cores. For example, if we were to have 100 threads and only four CPU cores, then the kernel would use preemptive multitasking to time-slice between 25 threads on each core.

4.4.5 Processes

A *process* is the operating system's way of managing a *running instance* of a program contained in an *executable file* (.exe on Windows, .elf on Linux). A process only exists while its program is actually running—when an instance of a program exits, is killed, or crashes, the OS destroys the process associated with that instance. Multiple processes can be running on a computer system at any given time. This might include multiple instances of the *same* program.

Programmers interact with processes via an API provided by the operating system. The details of this API differ from OS to OS, but the key concepts are roughly the same across all of them. A complete discussion of any one operating system's process API is beyond the scope of this book, but for the purposes of illustrating the concepts we'll focus primarily on the API style of UNIX-like operating systems such as Linux, BSD and MacOS. But we'll make note of situations in which Windows or game console operating systems deviate significantly from the core ideas of a UNIX-like process API.

4.4.5.1 Anatomy of a Process

Under the hood, a process consists of:

- a *process id (PID)* that uniquely identifies the process within the operating system;
- a set of *permissions*, such as which user “owns” each process and to which user group it belongs;
- a reference to the process's *parent process*, if any,
- a *virtual memory space* containing the process's “view” of physical memory (see Section 4.4.5.2 for more information);

- the values of all defined *environment variables*;
- the set of all open *file handles* in use by the process;
- the current *working directory* for the process,
- resources for managing *synchronization* and *communication* between processes in the system, such as message queues, pipes, and semaphores;
- one or more *threads*.

A *thread* encapsulates a running instance of a single stream of machine language instructions. By default, a process contains a single thread. But as we'll discuss in depth in Section 4.4.6, more than one thread can be created within a process, allowing more than one instruction stream to run *concurrently*. The kernel schedules all of the threads in the system (from all currently-running processes) to run on the available cores. It uses preemptive multitasking to time-slice between threads when there are more threads than there are cores.

We should stress here that threads are the *fundamental unit of program execution* within an operating system, not processes. A process merely provides an *environment* within which its thread(s) can run, including a virtual memory map and a set of *resources* that are used by and shared between all threads within that process. Whenever a thread is scheduled to run on a core, its process becomes active, and that process's resources and environment become available for use by the thread while it runs. So when we say that a *thread* is running on a core, remember that it is always doing so within the *context* of exactly one process.

4.4.5.2 Virtual Memory Map of a Process

You'll recall from Section 3.5.2 that a program generally never works directly with physical memory addresses.³ Rather, the program accesses memory in terms of *virtual addresses*, and the CPU and operating system cooperate to *remap* these virtual addresses into physical addresses. We said that the remapping of virtual to physical addresses happens in terms of contiguous blocks of addresses called *pages*, and that a *page table* is used by the OS to map virtual page indices to physical page indices.

Every process has its own virtual page table. This means that every process has its own custom *view* of memory. This is one of the primary ways in which the operating system provides a secure and stable execution environment. Two processes cannot corrupt each other's memory, because the physical pages owned by one process are simply not mapped into the other process's address space (unless they explicitly share pages). Also, pages owned by the

³User programs always work in terms of virtual memory addresses, but the kernel can work directly with physical addresses.

kernel are protected from inadvertent or deliberate corruption by a user process because they are mapped to a special range of addresses known as *kernel space* which can only be accessed by code running in kernel mode.

A process's virtual page table effectively defines its *memory map*. The memory map typically contains:

- the text, data and BSS sections as read in from the program's executable file;
- a view of any shared libraries (DLLs, PRXs) used by the program;
- a *call stack* for each thread;
- a region of memory called the *heap* for dynamic memory allocation;
- possibly some pages of memory that are *shared* with other processes; and
- a range of *kernel space* addresses which are inaccessible to the process (but become accessible whenever a kernel call executes).

Text, Data and BSS Sections

When a program is first run, the kernel creates a process internally and assigns it a unique PID. It then sets up a virtual page map for the process—in other words, it creates the virtual address space of the process. It then allocates physical pages as necessary and maps them into the virtual address space by adding entries to the process's page table.

The kernel reads the executable file (text, data and BSS sections) into memory by allocating virtual pages and loading the data into them. This allows the program's code and global data to be “visible” within the process's virtual address space. The machine code in an executable file is actually *relocatable*, meaning that its addresses are specified as relative offsets rather than absolute memory addresses. These relative addresses are *fixed up* by the operating system, meaning they are converted back into real (virtual) addresses, prior to running the program. (For more on the format of an executable file, see Section 3.3.5.1.)

Call Stack

Every running thread needs a call stack (see Section 3.3.5.2). When a process is first run, the kernel creates a single default thread for it. The kernel allocates physical memory pages for this thread's call stack, and maps them into the process' virtual address space so the stack can be “seen” by the thread. The values of the stack pointer (SP) and base pointer (BP) are initialized to point to the bottom of the empty stack. Finally, the thread starts executing at the

entry point of the program. (In C/C++ this is typically `main()`, or `WinMain()` under Windows.)

Heap

Processes can allocate memory *dynamically* via `malloc()` and `free()` in C, or global `new` and `delete` in C++. These requests come from a region of memory called the *heap*. Physical pages of memory are allocated on demand by the kernel in order to fulfill dynamic allocation requests; these pages are mapped into the virtual address space of the process as memory is allocated by it, and pages whose contents have been completely freed are unmapped and returned to the system.

Shared Libraries

All non-trivial programs depend on external libraries. A library can be *statically* linked into a program, meaning that a *copy* of the library's code is placed into the executable file itself. Most operating systems also support the concept of *shared libraries*. In this case, the program contains only *references* to the library's API functions, not a copy of the library's machine code. Shared libraries are called *dynamic link libraries* (DLL) under Windows. On the PlayStation 4, the OS supports a kind of dynamically linked library called a PRX. (Interestingly, the name PRX comes from the PlayStation 3, where it stood for **P**PU **R**elocatable **E**xecutable, in reference to the main processor in the PS3 which was called the PPU.)

Shared libraries generally work as follows: The first time a shared library is needed by a process, the OS loads that library into physical memory, and maps a view of it into the process's virtual address space. The addresses of the functions and global variables provided by the shared library are patched into the program's machine code, allowing it to call them as if they had been statically linked into the executable.

The benefit of shared libraries only becomes evident when a second process is run that uses the same shared library. Rather than loading a *copy* of the library's code and global variables, the already-loaded physical pages are simply mapped into the virtual address space of the new process. This saves memory and speeds up the process of running all but the first process that uses a given shared library.

Shared libraries have other benefits, too. For example, a shared library can be updated, say to fix some bugs, and in theory all programs that use that shared library will immediately benefit (without having to be relinked and redistributed to users). That said, in practice updating shared libraries can

inadvertently cause compatibility problems amongst the programs that use them. This leads to a proliferation of different versions of each shared library within the system—a situation affectionately known as “DLL hell” amongst Windows developers. To work around these problems, Windows moved to a system of manifests that help to guarantee compatibility between shared libraries and the programs that use them.

Kernel Pages

On most operating systems, the address space of a process is actually divided into two large contiguous blocks—user space and kernel space. For example, on 32-bit Windows, user space corresponds to the address range from address 0x0 through 0x7FFFFFFF (the lower 2 GiB of the address space), while kernel space corresponds to addresses between 0x80000000 and 0xFFFFFFFF (the upper 2 GiB of the space). On 64-bit Windows, user space corresponds to the 8 TiB range of addresses from 0x0 through 0x7FFF'FFFFFFFF, and the gigantic 248 TiB range from 0xFFFFF0800'00000000 through 0xFFFFFFFF'FFFFFFFF is reserved for use by the kernel (although not all of it is actually used).

User space is mapped through a virtual page table that is unique to each process. However, kernel space uses a separate virtual page table that is shared between all processes. This is done so that all processes in the system have a consistent “view” of the kernel’s internal data.

Normally, user processes are prevented from accessing the kernel’s pages—if they try to do so, a page fault will occur and the program will crash. However, when a user process makes a system call, a *context switch* (see Section 4.4.6.5) is performed into the kernel. This puts the CPU into privileged mode, allowing the kernel to access the kernel space address ranges (as well as the virtual pages of the current process). The kernel runs its code in privileged mode, updates its internal data structures as necessary, and finally puts the CPU back into user mode and returns control to the user program. For more details on how user and kernel space memory mapping works under Windows, search for “virtual address spaces” on <https://docs.microsoft.com>.

It’s interesting (and a bit frightening!) to note that the recently-discovered “Meltdown” and “Spectre” exploits make use of a CPU’s *out-of-order* and *speculative* execution logic (respectively) to trick it into accessing data located in memory pages that are normally protected from a user-mode process. For more on these exploits and how operating systems are protecting themselves against them, see <https://meltdownattack.com/>.

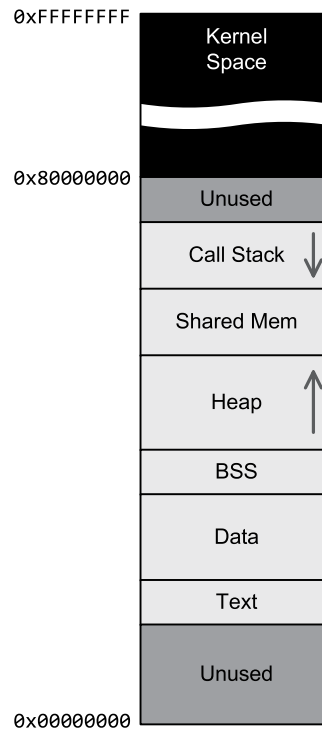


Figure 4.23. A process's memory map as it might look under 32-bit Windows.

Example Process Memory Map

Figure 4.23 depicts the memory map of a process as it might look under 32-bit Windows. All of the process's virtual pages are mapped into user space—the lower 2 GiB of the address space. The executable files' text, data and BSS segments are mapped at a low memory address, followed by the heap in a higher range, followed by any shared memory pages. The call stack is mapped at the high end of the user address space. Finally, the operating system's kernel pages are mapped into the upper 2 GiB of the address space.

The actual addresses for each of the regions in the memory map aren't predictable. This is partly because each program's segments are of different sizes and hence will map to different address ranges. Also, the numeric values of the addresses actually *change* between runs of the same executable program, thanks to a security measure known as *address space layout randomization* (ASLR).

4.4.6 Threads

A *thread* encapsulates a running instance of a single stream of machine language instructions. Each thread within a process is comprised of:

- a *thread id (TID)* which is unique within its process, but may or may not be unique across the entire operating system;
- the thread's *call stack*—a contiguous block of memory containing the stack frames of all currently-executing functions;
- the values of all special- and general-purpose *registers*⁴ including the *instruction pointer (IP)*, which points at the current instruction in the thread's instruction stream, the *base pointer (BP)* and *stack pointer (SP)* which define the current function's stack frame;
- a block of general-purpose memory associated with each thread, known as *thread local storage (TLS)*.

By default, a process contains a single *main thread* and hence executes a single instruction stream. This thread begins executing at the entry point of the program—typically the `main()` function. However, all modern operating systems are capable of executing more than one *concurrent* instruction stream within the context of a single process.

You can think of a thread as the *fundamental unit of execution* within the operating system. A thread provides the minimum resources required to execute an instruction stream—a call stack and a set of registers. The process merely provides the *environment* in which one or more threads execute. This is illustrated in Figure 4.24.

4.4.6.1 Thread Libraries

All operating systems that support multithreading provide a collection of system calls for creating and manipulating threads. A few portable thread libraries are also available, the best known of which are the IEEE POSIX 1003.1c standard thread library (pthread) and the C11 and C++11 standard thread libraries. The Sony PlayStation 4 SDK provides a set of thread functions prefixed with `sce` that map pretty much directly to the POSIX thread API.

The various thread APIs differ in their details, but all of them support the following basic operations:

1. *Create*. A function or class constructor that spawns a new thread.

⁴Technically a thread's execution context only encompasses the values of registers that are visible in *user mode*; it excludes the values of certain privileged-mode registers.

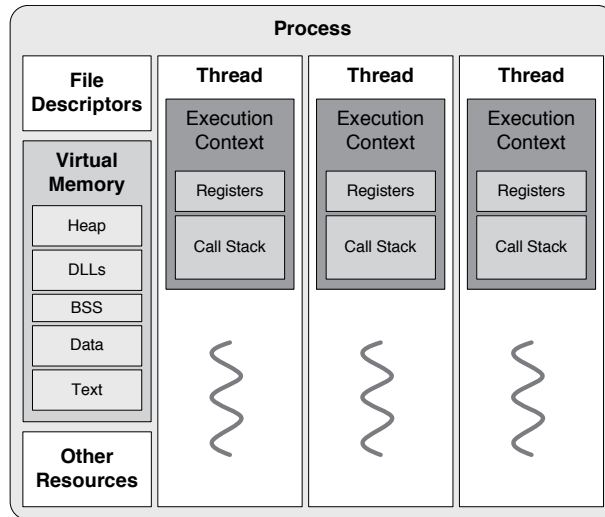


Figure 4.24. A process encapsulates the resources required to run one or more threads. Each thread encapsulates an *execution context* comprised of the contents of the CPU's registers and a call stack.

2. *Terminate*. A function that terminates the calling thread.
3. *Request to exit*. A function that allows one thread to request another thread to exit.
4. *Sleep*. A function that puts the current thread to sleep for a specified length of time.
5. *Yield*. A function that yields the remainder of the thread's time slice so other threads can get a chance to run.
6. *Join*. A function that puts the calling thread to sleep until another thread or group of threads has terminated.

4.4.6.2 Thread Creation and Termination

When an executable file is run, the process created by the OS to encapsulate it automatically contains a single thread, and execution of this thread begins at the *entry point* of the program—in C/C++ this is the special function `main()`. This “main thread” can spawn new threads if desired, by making calls to an operating system specific function such as `pthread_create()` (POSIX threads), `CreateThread()` (Windows), or by instantiating an instance of a

thread class such as `std::thread` (C++11). The new thread begins execution at an entry point function whose address is provided by the caller.

Once created, a thread will continue to exist until it terminates. The execution of a thread can terminate in a number of ways:

- It can end “naturally” by *returning* from its entry point function. (In the special case of the main thread, returning from `main()` not only ends the thread, but also ends the entire process.)
- It can call a function such as `pthread_exit()` to *explicitly* terminate its execution *before* having returned from its entry point function.
- It can be *killed* externally by another thread. In this case, the external thread makes a *request* to cancel the thread in question, but the thread may not respond immediately to the request, or it may ignore the request entirely. The cancelability of a thread is determined when that thread is created.
- It can be forcibly killed because its process has ended. (A process terminates when the main thread returns from the `main()` entry point function, when any thread calls `exit()` to kill the process explicitly, or when an external actor kills the process.)

4.4.6.3 Joining Threads

It’s common for one thread to spawn one or more child threads, do some useful work of its own, and then wait for the child threads to be done with their work before continuing. For example, let’s say the main thread wants to perform 1000 computations, and let’s assume further that this program is running on a quad-core machine. The most efficient approach would be to divide the work into four equally-sized chunks, and spawn four threads to do the processing in parallel. Once the computations are complete, let’s assume that the main thread wants to perform a checksum on the results. The resulting code might look something like this:

```
ComputationResult g_aResult[1000];

void Compute(void* arg)
{
    uintptr_t startIndex = (uintptr_t)arg;
    uintptr_t endIndex = startIndex + 250;
    for (uintptr_t i = startIndex; i < endIndex; ++i)
    {
        g_aResult[i] = ComputeOneResult(...);
    }
}
```

```

    }
}

void main()
{
    pthread_t  tid[4];

    for (int i = 0; i < 4; ++i)
    {
        const uintptr_t startIndex = i * 250;
        pthread_create(&tid[i], nullptr,
                      Compute, (void*)startIndex);
    }

    // perhaps do some other useful work...

    // wait for computations to be done
    for (int i = 0; i < 4; ++i)
    {
        pthread_join(&tid[i], nullptr);
    }

    // all threads are done, so we can do our checksum
    unsigned checksum = Sha1(g_aResult,
                             1000*sizeof(ComputationResult));

    // ...
}

```

4.4.6.4 Polling, Blocking and Yielding

Normally a thread runs until it terminates. But sometimes a running thread needs to wait for some future event to occur. For example, a thread might need to wait for a time-consuming operation to complete, or for some resource to become available. In such a situation, we have three options:

1. The thread can *poll*,
2. it can *block*, or
3. it can *yield* while polling.

Polling

Polling involves a thread sitting in a tight loop, waiting for a condition to be-

come true. It's a bit like kids in the back seat on a road trip, repeatedly asking, "Are we there yet? Are we there yet?" Here's an example:

```
// wait for condition to become true
while (!CheckCondition())
{
    // twiddle thumbs
}

// the condition is now true and we can continue...
```

It should be obvious that this approach, while simple, has the potential of burning CPU cycles unnecessarily. This approach is sometimes called a *spin-wait* or a *busy-wait*.

Blocking

If we expect our thread to wait for a relatively long period of time for a condition to become true, busy-waiting is not a good option. Ideally we'd like to put our thread to sleep so that it doesn't waste CPU resources, and rely on the kernel to wake it back up when the condition becomes true at some future time. This is called *blocking* the thread.

A thread blocks by making a special kind of operating system call known as a *blocking function*. If the condition is already true at the moment a blocking function is called, the function won't actually block—it will simply return immediately. But if the condition is false, the kernel will put the thread to sleep, and add the thread and the condition on which it is waiting into a table. Later, when the condition becomes true, the kernel uses this internal table to identify and wake up any threads that are waiting on that condition.

There are all sorts of OS functions that block. Here are a few examples:

- *Opening a file.* Most functions that open a file such as `fopen()` will block the calling thread until the file has actually been opened (which may take hundreds or even thousands of cycles). Some functions, like `open()` under Linux, offer a non-blocking option (`O_NONBLOCK`) to support asynchronous file I/O.
- *Explicit sleeping.* Some functions explicitly put the calling thread to sleep for a specified length of time. Variants include `usleep()` (Linux), `Sleep()` (Windows) `std::this_thread::sleep_until()` (C++11 standard library) and `pthread_sleep()` (POSIX threads).
- *Joining with another thread.* A function such as `pthread_join()` blocks the calling thread until the thread being waited on has terminated.

- *Waiting for a mutex lock.* Functions like `pthread_mutex_wait()` attempt to obtain an exclusive lock on a resource via an operating system object known as a *mutex* (see Section 4.6). If no other thread holds the lock, the function grants the lock to the calling thread and returns immediately; otherwise, the calling thread is put to sleep until the lock can be obtained.

Operating system calls aren't the only functions that can block. Any user-space function that ultimately calls a blocking OS function is itself considered to be a blocking function. It's a good idea to document such a function, so that the programmers who use it will know that it has the potential to block.

Yielding

This technique falls part-way between polling and blocking. The thread polls the condition in a loop, but on each iteration it relinquishes the remainder of its time slice by calling `pthread_yield()` (POSIX), `Sleep(0)` or `SwitchToThread()` (Windows), or an equivalent system call.

Here's an example:

```
// wait for condition to become true
while (!CheckCondition())
{
    // yield the remainder of my time slice
    pthread_yield(NULL);
}

// the condition is now true and we can continue...
```

This approach tends to result in fewer wasted cycles and better power consumption than a pure busy-wait loop.

Yielding the CPU still involves a kernel call, and is therefore quite expensive. Some CPUs provide a lightweight “pause” instruction. (For example, on an Intel x86 ISA with SSE2, the `_mm_pause()` intrinsic emits such an instruction.) This kind of instruction reduces the power consumption of a busy-wait loop by simply waiting for the CPU's instruction pipeline to empty out before allowing execution to continue:

```
// wait for condition to become true
while (!CheckCondition())
{
    // Intel SSE2 only:
    // reduce power consumption by pausing for ~40 cycles
    _mm_pause();
}
```

```
}  
  
// the condition is now true and we can continue...
```

See <https://software.intel.com/en-us/comment/1134767> and <http://software.intel.com/en-us/forums/topic/309231> for an in-depth discussion of how and why to use a pause instruction in a busy-wait loop.

4.4.6.5 Context Switching

Every thread maintained by the kernel exists in one of three states:⁵

- *Running*. The thread is actively running on a core.
- *Runnable*. The thread is able to run, but it is waiting to receive a time slice on a core.
- *Blocked*. The thread is asleep, waiting for some condition to become true.

A *context switch* occurs whenever the kernel causes a thread to transition from one of these states to another.

A context switch always happens in privileged mode on the CPU—in response to the hardware interrupt that drives preemptive multitasking (i.e., transitions between Running and Runnable), in response to an explicit blocking kernel call made by a running thread (i.e., a transition from Running or Runnable to Blocked), or in response to a waited-on condition becoming true, thus “waking” a sleeping thread (i.e., transitioning it from Blocked to Runnable). The kernel’s thread state machine is illustrated in Figure 4.25.

When a thread is in the Running state, it is actively making use of a CPU core. The core’s registers contain information pertinent to the execution of that thread, such as its instruction pointer (IP), stack and base pointers (SP and BP), and the contents of various general-purpose registers (GPRs). The thread also maintains a call stack, which stores local variables and return addresses for the currently-running function and the entire stack of functions that ultimately called it. Together, this information is known as the thread’s *execution context*.

Whenever a thread transitions away from the Running state to either Runnable or Blocked, the contents of the CPU’s registers are saved to a memory block that has been reserved for the thread by the kernel. Later, when a Runnable thread transitions back to the Running state, the kernel repopulates the CPU’s registers with that thread’s saved register contents.

⁵Some operating systems make use of additional states, but such states are implementation details that we can safely ignore for our purposes.

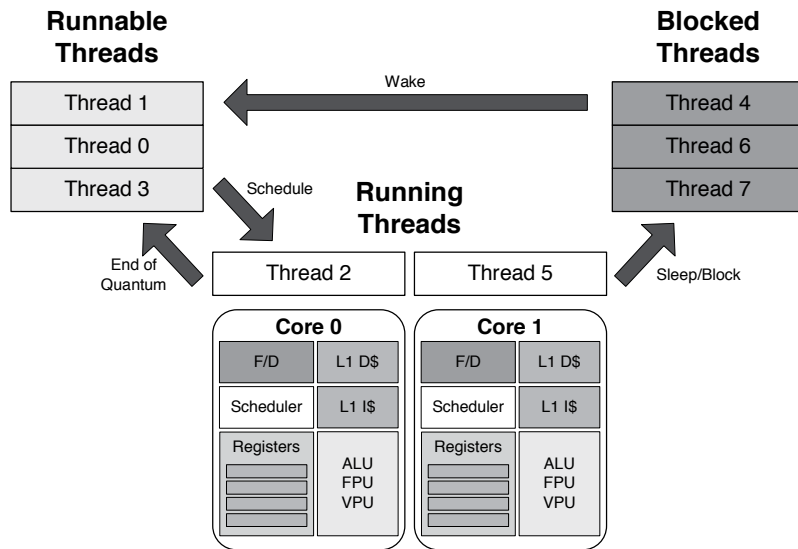


Figure 4.25. Every thread can be in one of three states: Running, Runnable or Blocked.

We should note here that a thread's call stack need not be saved or restored explicitly during a context switch. This is because each thread's call stack already resides in a distinct region within its process' virtual memory map. The act of saving and restoring the contents of the CPU's registers includes saving and restoring the stack and base pointers (SP and BP), and thereby effectively saves and restores the thread's call stack "for free."

During a context switch, if the incoming thread resides in a different process from that of the outgoing thread, the kernel also needs to save off the state of the outgoing process' virtual memory map, and set up the virtual memory map of the incoming process. You'll recall from Section 3.5.2 that a virtual memory map is defined by a virtual page table. The act of saving and restoring a virtual memory map therefore involves saving and restoring a pointer to this page table, which is usually maintained in a special privileged CPU register. The translation lookaside buffer (TLB) must also be flushed whenever an inter-process context switch occurs (see Section 3.5.2.4). These additional steps make context switching between processes more expensive than context switching between threads within a single process.

4.4.6.6 Thread Priorities and Affinity

For the most part, the kernel handles the job of scheduling threads to run on the available cores in the machine. However, programmers do have two ways

to affect how threads are scheduled: *priority* and *affinity*.

A thread's priority controls how it is scheduled relative to other Runnable threads in the system. Higher-priority threads generally take precedence over lower-priority threads. Different operating systems offer different numbers of priority levels. For example, Windows threads can belong to one of six priority classes, and there are seven distinct priority levels within each class. These two values are combined to produce a total of 32 distinct "base priorities" which are used when scheduling threads.

The simplest thread scheduling rule is this: As long as at least one higher-priority Runnable thread exists, no lower-priority threads will be scheduled to run. The idea behind this approach is that most threads in the system will be created at some default priority level, and hence will share the processing resources fairly. But once in a while, a higher-priority thread can become Runnable. When it does, it runs as close to immediately as possible, hopefully exits after a relatively short period of time, and thereby returns control to all of the lower-priority threads.

Such a simple priority-based scheduling algorithm can lead to a situation in which a small number of high-priority threads run continually, thereby preventing any lower-priority threads from running. This is known as *starvation*. Some operating systems attempt to mitigate the ill effects of starvation by introducing exceptions to the simple scheduling rule that aim to give at least some CPU time to starving lower-priority threads.

Another way in which programmers can control thread scheduling is via a thread's *affinity*. This setting requests that the kernel either lock a thread to a particular core, or that it should at least *prefer* one or more cores over the others when scheduling the thread.

4.4.6.7 Thread Local Storage

We said that all threads within a process share the process's resources, including its virtual memory space. There is one exception to this rule—each thread is given a private memory block known as *thread local storage* (TLS). This allows threads to keep track of data that shouldn't be shared with other processes. For example, each thread might maintain a private memory allocator. We can think of the TLS memory block as being a part of the thread's execution context.

In practice, TLS memory blocks are usually visible to all threads within a process. They're typically not protected, the way operating system virtual memory pages are. Instead, the OS grants each thread its own TLS block, all mapped into the process's virtual address space at different numerical addresses, and a system call is provided that allows any one thread to obtain

	ID	Managed ID	Category	Name	Location	Priority
▲	SeqDotNetMandelbrot.vshost.exe (id = 54424) : C:\Users\Pavel\Documents\MatrixWorkshops\SeqDotNetMandelbrot\SeqDotNetMa					
🚩	0x00	0x00	Unknown Thread	[Thread Destroyed]	<not available>	
🚩	0x00005E8C	0x00	Worker Thread	<No Name>	<not available>	Highest
🚩	0x0000F6EC	0x00000003	Worker Thread	<No Name>	<not available>	Normal
🚩	0x00008B50	0x00000006	Worker Thread	vshost.RunParkingWindow	▼ [Managed tc	Normal
🚩	0x00008BF0	0x00000008	Main Thread	Main Thread	▼ DotNetManr	Normal
🚩	0x0000EC54	0x00000007	Worker Thread	.NET SystemEvents	▼ [Managed tc	Normal
🚩	0x000067F8	0x00000009	Worker Thread	<No Name>	▼ DotNetManr	Normal
🚩	0x000107C4	0x0000000A	Worker Thread	<No Name>	▼ DotNetManr	Normal
🚩	0x00009B44	0x0000000B	Worker Thread	<No Name>	▼ DotNetManr	Normal
🚩	0x00007158	0x0000000C	Worker Thread	<No Name>	▼ DotNetManr	Normal
🚩	0x00007008	0x0000000D	Worker Thread	<No Name>	▼ DotNetManr	Normal
🚩	0x00004DA4	0x0000000F	Worker Thread	<No Name>	<not available>	Normal
🚩	0x000039B8	0x0000000E	Worker Thread	<No Name>	<not available>	Normal
🚩	0x0001171C	0x00000010	Worker Thread	<No Name>	▼ DotNetManr	Normal

Figure 4.26. The Threads window in Visual Studio is the primary interface for debugging multithreaded programs.

the address of its private TLS block.

4.4.6.8 Thread Debugging

All good debuggers nowadays provide tools for debugging multithreaded applications. In Microsoft Visual Studio, the *Threads Window* is the central tool for this purpose. Whenever you break into the debugger, this window lists all the threads currently in existence within the application. Double-clicking on a thread makes its execution context active within the debugger. Once a thread's context has been activated, you can walk up and down its call stack via the Call Stack window, and view local variables within each function's scope via the Watch windows. This works even if the thread is in its Runnable or Blocked state. The Visual Studio Threads window is shown in Figure 4.26.

4.4.7 Fibers

In preemptive multitasking, thread scheduling is handled automatically by the kernel. This is often convenient, but sometimes programmers find it desirable to have control over the scheduling of workloads in their programs. For example, when implementing a *job system* for a game engine (discussed in Section 8.6.4), we might want to allow jobs to explicitly yield the CPU to other jobs, without worrying about the possibility of preemption “pulling the rug out” from under our jobs as they run. In other words, sometimes we want to use *cooperative* rather than preemptive multitasking.

Some operating systems provide just such a cooperative multitasking mechanism: They are known as *fibers*. A fiber is a lot like a thread, in that it represents a running instance of a stream of machine language instructions. A fiber has a call stack and register state (an execution context), just like a thread. However, the big difference is that a fiber is never scheduled directly by the kernel. Instead, fibers run within the context of a thread, and are scheduled cooperatively, by each other.

In this section, we'll talk about Windows fibers specifically. Some other operating systems, such as Sony's PlayStation 4 SDK, provide very similar fiber APIs.

4.4.7.1 Fiber Creation and Destruction

How do we convert a thread-based process into a fiber-based one? Every process starts with a single thread when it first runs; hence processes are thread-based by default. When a thread calls the function `ConvertThreadToFiber()`, a new fiber is created within the context of the calling thread. This "bootstraps" the process so that it can create and schedule more fibers. Other fibers are created by calling `CreateFiber()` and passing it the address of a function that will serve as its entry point. Any running fiber can cooperatively schedule a different fiber to run within its thread by calling `SwitchToFiber()`. When a fiber is no longer needed, it can be destroyed by calling `DeleteFiber()`.

4.4.7.2 Fiber States

A fiber can be in one of two states: Active or Inactive. When a fiber is in its Active state, it is assigned to a thread, and executes on its behalf. When a fiber is in its Inactive state, it is sitting on the sidelines, not consuming the resources of any thread, just waiting to be activated. Windows calls an Active fiber the "selected" fiber for a given thread.

An Active fiber can deactivate itself and make another fiber active by calling `SwitchToFiber()`. This is the only way that fibers can switch between the Active and Inactive states.

Whether or not an Active fiber is actively executing on a CPU core is determined by the state of its enclosing thread. When an Active fiber's thread is in the Running state, that fiber's machine language instructions are being executed on a core. When an Active fiber's thread is in the Runnable or Blocked state, its instructions of course cannot execute, because the entire thread is sitting on the sidelines, either waiting to be scheduled on a core or waiting for a condition to become true.

It's important to understand that fibers don't themselves have a Blocked state, the way threads do. In other words, it's not possible to put a fiber to sleep waiting on a condition. Only its thread can be put to sleep. Because of this restriction, whenever a fiber needs to wait for a condition to become true, it either busy-waits or it calls `SwitchToFiber` in order to yield control to another fiber while it waits. Making a blocking OS call from within a fiber is usually a pretty big no-no. Doing so would put the fiber's enclosing thread to sleep, thereby preventing that fiber from doing anything—including scheduling other fibers to run cooperatively.

4.4.7.3 Fiber Migration

A fiber can migrate from thread to thread, but only by passing through its Inactive state. As an example, consider a fiber `F` that is running within the context of thread `A`. Fiber `F` calls `SwitchToFiber(G)` to activate a different fiber named `G` inside thread `A`. This puts fiber `F` into its Inactive state (meaning it is no longer associated with any thread). Now let's assume that another thread named `B` is running fiber `H`. If fiber `H` calls `SwitchToFiber(F)`, then fiber `F` has effectively migrated from thread `A` to thread `B`.

4.4.7.4 Debugging with Fibers

Because fibers are provided by the OS, debugging tools and profiling tools should be able to “see” them, just the way they can “see” threads. For example, when debugging on the PS4 using SN Systems' Visual Studio debugger plugin for Clang, fibers automatically show up in the Threads window as if they were threads. You can double-click a fiber to activate it within the Watch and Call Stack windows, and then walk up and down its call stack just as you normally would with a thread.

If you're considering using fibers in your game engine, it's a good idea to check out your debugger's capabilities on your target platform before you commit a lot of time and effort to a fiber-based design. If your debugger and/or target platform doesn't provide good tools for debugging fibers, that could be a deal breaker.

4.4.7.5 Further Reading on Fibers

You can read more about Windows fibers here: [https://msdn.microsoft.com/en-us/library/windows/desktop/ms682661\(v=vs.85\).aspx](https://msdn.microsoft.com/en-us/library/windows/desktop/ms682661(v=vs.85).aspx).

4.4.8 User-Level Threads and Coroutines

Both threads and fibers tend to be rather “heavy weight” because these facilities are provided by the kernel. This implies that most functions that you’d call to manipulate threads or fibers involve a context switch into kernel space—not a cheap operation. But there are lighter-weight alternatives to threads and fibers. These mechanisms allow programmers to code in terms of multiple independent flows of control, each with its own execution context, but without the high cost of making kernel calls. Collectively, these facilities are known as *user-level threads*.

User-level threads are implemented entirely in user space. The kernel knows nothing about them. Each user-level thread is represented by an ordinary data structure that keeps track of the thread’s id, possibly a human-readable name, and execution context information (the contents of CPU registers and a call stack). A user-level thread library provides API functions for creating and destroying threads, and context switching between them. Each user-level thread runs within the context of a “real” thread or fiber that has been provided by the operating system.

The trick to implementing a user-level thread library is figuring out how to implement a context switch. If you think about it, a context switch mostly boils down to swapping the contents of the CPU’s registers. After all, the registers contain all of the information needed to describe a thread’s execution context—including the instruction pointer and the call stack. So by writing some clever assembly language code, it’s possible to implement a context switch. And once you have a context switch, the rest of your user-level thread library is nothing more than data management.

User-level threads aren’t supported very well in C and C++, but some portable and non-portable solutions do exist. POSIX provided a collection of functions for managing lightweight thread execution contexts via its `ucontext.h` header file (<https://en.wikipedia.org/wiki/Setcontext>), but this API has since been deprecated. The C++ Boost library provides a portable user-level thread library. (Search for “context” on <http://www.boost.org/> for documentation on this library.)

4.4.8.1 Coroutines

Coroutines are a particular type of user-level thread that can prove very useful for writing inherently asynchronous programs, like web servers—and games! A coroutine is a generalization of the concept of a subroutine. Whereas a subroutine can only exit by returning control to its caller, a coroutine can also exit by *yielding* to another coroutine. When a coroutine yields, its execution

context is maintained in memory. The next time the coroutine is *called* (by being yielded *to* by some other coroutine) it continues from where it left off.

Subroutines call each other in a heirarchical fashion. Subroutine A calls B, which calls C, which returns to B, which returns to A. But coroutines call each other symmetrically. Coroutine A can yield to B, which can yield to A, ad infinitum. This back and forth calling pattern doesn't lead to an infinitely deepening call stack, because each coroutine maintains its own private execution context (call stack and register contents). Thus yielding from coroutine A to coroutine B acts more like a context switch between threads than like a function call. But because coroutines are implemented with *user-level* threads, these context switches are very efficient.

Here's a pseudocode example of a system in which one coroutine continually produces data that is consumed by another coroutine:

```
Queue g_queue;

coroutine void Produce()
{
    while (true)
    {
        while (!g_queue.IsFull())
        {
            CreateItemAndAddToQueue(g_queue);
        }
        YieldToCoroutine(Consume);

        // continues from here on next yield...
    }
}

coroutine void Consume()
{
    while (true)
    {
        while (!g_queue.IsEmpty())
        {
            ConsumeItemFromQueue(g_queue);
        }
        YieldToCoroutine(Produce);

        // continues from here on next yield...
    }
}
```

Coroutines are most often provided by high-level languages like Ruby, Lua

and Google's Go. It's also possible to use coroutines in C or C++. The C++ Boost library provides a solid implementation of coroutines, but Boost requires you to compile and link against a pretty huge codebase. If you want something leaner, you may want to try rolling your own coroutine library. The following blog post by Malte Skarupke demonstrates that doing so isn't quite as onerous a task as you might at first imagine: <https://probablydance.com/2013/02/20/handmade-coroutines-for-windows/>.

4.4.8.2 Kernel Threads versus User Threads

The term “kernel thread” has two very different meanings, and this can become a major source of confusion as you read more about multithreading. So let's demystify the term. The two definitions are as follows:

1. On Linux, a “kernel thread” is a special kind of thread created for internal use by the kernel itself, which runs only while the CPU is in privileged mode. The kernel also creates threads for use by processes (via an API such as `pthread` or C++11's `std::thread`). These threads run in user space within the context of a process. In this sense of the term, any thread that runs in privileged mode is a kernel thread, and any thread that runs in user mode (in the context of a single-threaded or multithreaded process) is a “user thread.”
2. The term “kernel thread” can also be used to refer to any thread that is *known to* and *scheduled by* the kernel. Using this definition, a kernel thread can execute in either kernel space or user space, and the term “user thread” only applies to a flow of control that is managed entirely by a user-space program *without* the kernel being involved at all, such as a coroutine.

Using definition #2, a fiber blurs the line between “kernel thread” and “user thread.” On the one hand, the kernel is aware of fibers and maintains a separate call stack for each one. On the other hand, a fiber is not scheduled by the kernel—it can run only when another fiber or thread explicitly hands control to it via a call such as `SwitchToFiber()`.

4.4.9 Further Reading on Processes and Threads

We've covered the basics of processes, threads and fibers in the preceding sections, but really we've only just scratched the surface. For more information, check out some of the following websites: